COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	07/22/26	**DATE SUBMITTED:**	07/22/26
NAME:	Reyes, Cyrill Joferson B	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 3 Singly Linked List

1. PROCEDURE OUTPUT

What procedures did you perform?

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

CMakeLists.txt | main.cpp | singly_ll.h

```cpp
#include <iostream>
#include "singly_ll.h"

using namespace std;

int main()
{
    SinglyList<char>* head = nullptr;
    SinglyList<char>* second = nullptr;
    SinglyList<char>* third = nullptr;
    SinglyList<char>* fourth = nullptr;
    SinglyList<char>* fifth = nullptr;
    SinglyList<char>* sixth = nullptr;

    head = new SinglyList<char>;
    second = new SinglyList<char>;
    third = new SinglyList<char>;
    fourth = new SinglyList<char>;
    fifth = new SinglyList<char>;
    sixth = new SinglyList<char>;

    head->data = 'C';
    second->data = 'Y';
    third->data = 'R';
    fourth->data = 'I';
    fifth->data = 'L';
    sixth->data = 'L';
```

```
28
29      head->next = second;
30      second->next = third;
31      third->next = fourth;
32      fourth->next = fifth;
33      fifth->next = sixth;
34      sixth->next = nullptr;
35
36      cout << "Testing the ListTraversal Function:" << endl;
37      ListTraversal(head);
38
39      cout << "\nTesting the Insert at Head Function:" << endl;
40      sllInsertHead( newData: 'Q', &head);
41      ListTraversal(head);
42
43      cout << "\nTesting the General Insert Function:" << endl;
44      sllGeneralInsert( newData: 'X', head->next->next);
45      ListTraversal(head);
46
47      cout << "\nTesting the Insert at End Function:" << endl;
48      sllInsertEnd( newData: 'U', &head);
49      ListTraversal(head);
50
51      cout << "\nTesting the Delete Node Function:" << endl;
52      sllDeleteNode( findData: 'X', &head);
53      ListTraversal(head);
```

f main

CMakeLists.txt | main.cpp | singly_ll.h | c_cpp_properties.jso

```
6   int main()
42  {
43      cout << "\nTesting the General Insert Function:" << endl;
44      sllGeneralInsert( newData: 'X', head->next->next);
45      ListTraversal(head);
46
47      cout << "\nTesting the Insert at End Function:" << endl;
48      sllInsertEnd( newData: 'U', &head);
49      ListTraversal(head);
50
51      cout << "\nTesting the Delete Node Function:" << endl;
52      sllDeleteNode( findData: 'X', &head);
53      ListTraversal(head);
54
55      sllDeleteNode( findData: 'Q', &head);
56      ListTraversal(head);
57
58      sllDeleteNode( findData: 'U', &head);
59      ListTraversal(head);
60
61      return 0;
62  }
```

main

CMakeLists.txt | main.cpp | singly_ll.h

```cpp
#ifndef SINGLY_LL_H
#define SINGLY_LL_H

#include <iostream>
using namespace std;

template <typename T>
class SinglyList
{
public:
    T data;
    SinglyList<T>* next;
};

template <typename T>
void ListTraversal(SinglyList<T>* node)
{
    while(node != nullptr)
    {
        cout << node->data;

        if(node->next != nullptr)
            cout << "->";

        node = node->next;
    }

```

CMakeLists.txt | main.cpp | singly_ll.h | c_cpp_properties.json

```
15  template <typename T>
16  void ListTraversal(SinglyList<T>* node)
28      cout << endl;
29  }
30
31  template <typename T>
32  void sllInsertHead(T newData, SinglyList<T>** currentHead)
33  {
34      auto* newNode = new SinglyList<T>;
35
36      newNode->data = newData;
37      newNode->next = *currentHead;
38
39      *currentHead = newNode;
40  }
41
42  template <typename T>
43  void sllGeneralInsert(T newData, SinglyList<T>* prevNode)
44  {
45      if(prevNode == nullptr)
46      {
47          cout << "Previous Node cannot be NULL!" << endl;
48          return;
49      }
50
51      auto* newNode = new SinglyList<T>;
52
```

CMakeLists.txt | main.cpp | singly_ll.h | c_cpp_properties

```
42      template <typename T>
43      void sllGeneralInsert(T newData, SinglyList<T>* prevNode
53          newNode->data = newData;
54          newNode->next = prevNode->next;
55
56          prevNode->next = newNode;
57      }
58
59      template <typename T>
60      void sllInsertEnd(T newData, SinglyList<T>** head)
61      {
62          auto* newNode = new SinglyList<T>;
63
64          newNode->data = newData;
65          newNode->next = nullptr;
66
67          if(*head == nullptr)
68          {
69              *head = newNode;
70              return;
71          }
72
73          SinglyList<T>* currentNode = *head;
74
75          while(currentNode->next != nullptr)
76          {
77              currentNode = currentNode->next;
```

```
78          }
79
80          currentNode->next = newNode;
81      }
82
83      template <typename T>
84      void sllDeleteNode(T findData, SinglyList<T>** head)
85      {
86          SinglyList<T>* currNode = *head;
87          SinglyList<T>* prevNode = nullptr;
88
89          while(currNode != nullptr && currNode->data != findData)
90          {
91              prevNode = currNode;
92              currNode = currNode->next;
93          }
94
95          if(currNode == nullptr)
96              return;
97
98          if(prevNode == nullptr)
99              *head = currNode->next;
100         else
101             prevNode->next = currNode->next;
102
```

CMakeLists.txt | main.cpp | singly_ll.h | c_cpp_properties.json

```
83      template <typename T>
84      void sllDeleteNode(T findData, SinglyList<T>** head)
87          SinglyList<T>* prevNode = nullptr;
88
89          while(currNode != nullptr && currNode->data != findData)
90          {
91              prevNode = currNode;
92              currNode = currNode->next;
93          }
94
95          if(currNode == nullptr)
96              return;
97
98          if(prevNode == nullptr)
99              *head = currNode->next;
100         else
101             prevNode->next = currNode->next;
102
103         delete currNode;
104     }
105
106     #endif #ifndef SINGLY_LL_H
```

```
"C:\Users\Joyce Reyes\Documents\CPE21S4 - Reyes\cmake-build-debug\CPE21S4___Reyes.exe"
Testing the ListTraversal Function:
C->Y->R->I->L->L

Testing the Insert at Head Function:
Q->C->Y->R->I->L->L

Testing the General Insert Function:
Q->C->Y->X->R->I->L->L

Testing the Insert at End Function:
Q->C->Y->X->R->I->L->L->U

Testing the Delete Node Function:
Q->C->Y->R->I->L->L->U
C->Y->R->I->L->L->U
C->Y->R->I->L->L

Process finished with exit code 0
```

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.



Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.